SAVITRIBAI PHULE PUNE UNIVERSITY

A Framework for Monitoring, Detecting, and Adaptive Response to Data Drift in Machine Learning Systems

A Dissertation Under the Guidance of
Dr. Omkar Barve
By
SANSKRUTI JOSHI

Submitted in partial fulfillment of the
requirements for the degree of
MSC. DATA SCIENCE

May 2026

Accepted by the University

Date HEAD OF THE DEPARTMENT

© Copyright 2026 by
Sanskruti Joshi
All Rights Reserved

BONAFIDE CERTIFICATE



Certified that this dissertation entitled:

A Framework for Monitoring, Detecting, and Adaptive Response to Data Drift in Machine Learning Systems

is the bonafide work of SANSKRUTI JOSHI, a candidate for the degree of MSc Data Science, who carried out the project work under my supervision.

Date

GUIDE Dr.Omkar Barve

HEAD OF THE DEPARTMENT

Department of
Technology
Savitribai Phule Pune
University, Pune.

To my family, whose constant support and encouragement made this work possible.

To my mentors and teachers, for their guidance and inspiration throughout this journey.

Acknowledgements

I would like to express my sincere gratitude to **Dr. Omkar Barve** for his valuable guidance, continuous support, and encouragement throughout the development of this dissertation. His insights and suggestions greatly contributed to the successful completion of this work.

I extend my heartfelt thanks to the faculty members of the **Department of Technology, Savitribai Phule Pune University**, for providing the necessary academic environment and resources for this research.

I am deeply grateful to my family and friends for their constant motivation, understanding, and unwavering support throughout this journey.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this dissertation

Abstract

Machine learning models deployed in real-world environments are highly susceptible to performance degradation due to changes in data distribution over time, commonly referred to as data drift. This dissertation presents a framework for monitoring, detecting, and adaptively responding to data drift in machine learning systems.

The proposed framework integrates statistical drift detection techniques, including the Kolmogorov–Smirnov (KS) test and Population Stability Index (PSI), to identify distributional changes in features. Controlled experiments are conducted to simulate both feature drift and label drift, enabling systematic evaluation of model robustness under changing conditions.

The impact of drift on model performance is analyzed using evaluation metrics such as accuracy, precision, recall, and F1-score. Experimental results demonstrate that drift significantly affects model behavior, particularly reducing recall and overall model reliability. To address this issue, an adaptive retraining mechanism is incorporated, where retraining is triggered automatically when drift exceeds predefined thresholds.

Results show that retraining helps mitigate performance degradation and improves model adaptability under drifted conditions, although baseline performance may not always be fully restored due to changes in underlying data characteristics. The study highlights the importance of continuous monitoring and adaptive maintenance strategies for sustaining reliable machine learning systems in dynamic environments.

Keywords: Data Drift, Machine Learning Monitoring, KS Test, Population Stability Index (PSI), Model Retraining, Performance Degradation

Contents

1 Introduction **1**

1.1 Project Overview 1

1.2 Importance of the Study 2

1.3 Research Objective 3

1.3.1 Data Drift in Machine Learning Systems. 4

1.3.2 Scope of the Proposed Framework 4

2 Literature Review **5**

2.1 Data Drift in Machine Learning Systems 5

2.2 Statistical Drift Detection Techniques 6

2.2.1 Kolmogorov–Smirnov (KS) Test 7

2.2.2 Population Stability Index (PSI) 8

2.3 MLOps and Model Monitoring 8

2.4 Adaptive Retraining Approaches 9

3 Project Schema **11**

3.1 Proposed Framework Architecture 11

3.2 Drift Detection 12

3.3 Adaptive Retraining Mechanism 14

3.4 Data Management and Experimental Setup 15

4 Results **20**

5 Conclusion and Future Scope **32**

Bibliography **36**

List of Tables

2.1 Comparative Analysis of Drift Detection Techniques 7

3.1 Drift Severity Classification based on PSI Thresholds 14

3.2 Major dataset features used in the experimental setup 19

4.1 Baseline Model Performance Metrics 20

4.2 Drift Detection Summary using KS Test and PSI 23

4.3 Comparative Analysis of Baseline, Drifted, and Retrained Model Performance 26

4.4 Summary of Key Experimental Observations 28

List of Figures

1.1 Proposed Framework Architecture for Drift Monitoring and Adaptive Retraining 3

4.1 Baseline vs Drifted Model Performance Comparison . . . 22

4.2 Feature Distribution Shift (Before vs After Drift) . . . 24

4.3 Model Performance Across Lifecycle Stages (Baseline → Drifted → Retrained) 26

4.4 Interactive Dashboard for Drift Monitoring using PSI Threshold Visualization 28

CHAPTER 1
Introduction

Machine learning systems are increasingly being deployed in real-world applications such as healthcare, finance, e-commerce, cybersecurity, and customer analytics. These systems are generally trained on historical data under the assumption that future incoming data will follow a similar distribution. However, in practical deployment environments, data distributions often change over time due to evolving user behavior, seasonal variations, economic fluctuations, policy changes, or system updates. This phenomenon is commonly referred to as **data drift**.

Data drift significantly impacts the reliability and effectiveness of machine learning models. A model that performs well during training may gradually lose predictive capability when exposed to new and changing data distributions. Such degradation can lead to inaccurate predictions, reduced business value, and poor decision-making. Therefore, continuous monitoring and maintenance of deployed machine learning systems have become essential components of modern Machine Learning Operations (MLOps).

This dissertation proposes a framework for monitoring, detecting, and adaptively responding to data drift in machine learning systems. The framework combines statistical drift detection techniques with performance evaluation and automated retraining mechanisms to maintain model reliability under dynamic conditions.

1.1 Project Overview

The proposed project presents an end-to-end framework for identifying and managing data drift in machine learning systems. The framework integrates drift simulation, statistical drift detection, performance monitoring, visualization, and adaptive retraining into a unified workflow.

The system initially trains a machine learning model using baseline data and evaluates its performance using metrics such as accuracy, precision, recall, and F1-score. Controlled drift is then introduced into the dataset to simulate real-world distributional changes. Statistical techniques such as the **Kolmogorov–Smirnov (KS)**

Test and **Population Stability Index (PSI)** are applied to identify and quantify drift in important features.

Once drift is detected, the framework evaluates the impact of drift on model performance. If the drift exceeds predefined thresholds, the system automatically triggers model retraining using updated data distributions. Interactive visualizations are incorporated using Plotly to support monitoring and interpretation of drift behavior.

The overall workflow of the proposed framework is illustrated in Figure 1.1.

1.2 Importance of the Study

As machine learning adoption continues to grow across industries, ensuring long-term reliability of deployed models has become a major challenge. Most machine learning models are developed and evaluated under static conditions, whereas real-world environments are dynamic and continuously evolving. Even minor changes in data distributions can significantly affect prediction quality.

Data drift is particularly critical in applications such as customer churn prediction, fraud detection, recommendation systems, and financial risk assessment, where inaccurate predictions can directly impact business decisions. In many cases, model degradation occurs gradually and remains undetected until significant performance loss has already occurred.

The proposed study addresses this challenge by providing a systematic framework for detecting drift and evaluating its practical impact on model performance. Unlike traditional approaches that focus only on model training, this work emphasizes continuous monitoring and adaptive maintenance of machine learning systems.

The framework also demonstrates how statistical drift metrics can be integrated with retraining strategies to improve model adaptability. This makes the study highly relevant to modern MLOps and production-level machine learning systems.

1.3 Research Objective

The primary objective of this research is to design and implement a framework capable of monitoring, detecting, and adaptively responding to data drift in machine learning systems. The study focuses on understanding how drift affects model performance and how automated retraining mechanisms can help maintain model reliability.

The proposed framework aims to achieve the following objectives:

1. To simulate different types of drift in machine learning datasets.
2. To detect distributional changes using statistical techniques such as KS Test and PSI.
3. To analyze the impact of drift on model evaluation metrics.
4. To implement a retraining mechanism triggered by drift severity.
5. To visualize drift behavior and performance degradation using interactive dashboards.

Figure 1.1 illustrates the overall system architecture and workflow of the proposed framework.

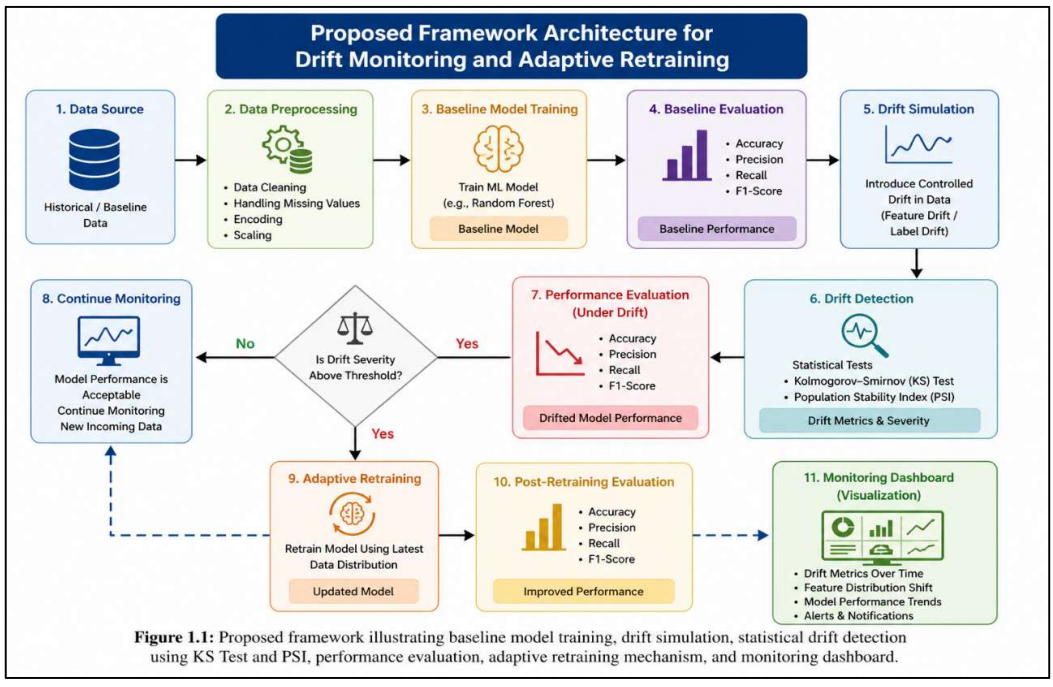


Figure 1.1: Proposed framework illustrating baseline model training, drift simulation, statistical drift detection using KS Test and PSI, performance evaluation, adaptive retraining mechanism, and monitoring dashboard.

1.3.1 Data Drift in Machine Learning Systems

Data drift refers to changes in the statistical properties of input data over time. These changes may occur gradually or suddenly and can significantly affect machine learning model performance. Drift may arise due to evolving customer behavior, market trends, operational changes, or environmental conditions.

The two major forms of drift considered in this work are:

- **Feature Drift (Covariate Drift):** Changes in the distribution of input features.
- **Label Drift:** Changes in the distribution of target variables or class labels.

Detecting drift is essential because machine learning models trained on historical data may no longer generalize effectively when exposed to updated distributions. Statistical methods such as KS Test and PSI help identify these changes and provide measurable indicators of drift severity.

1.3.2 Scope of the Proposed Framework

The scope of this dissertation is focused on the development of a practical framework for monitoring and managing drift in machine learning systems. The framework is implemented using Python-based data science tools and evaluated using controlled drift simulations.

The study primarily focuses on:

1. Detection of feature and label drift using statistical methods.
2. Evaluation of drift impact on classification performance metrics.
3. Automated retraining based on drift thresholds.
4. Visualization of drift patterns and model behavior through interactive dashboards.
5. Comparative analysis of baseline, drifted, and retrained model performance.

The proposed framework can be extended in future work for real-time monitoring systems, streaming data environments, and integration with production-level MLOps pipelines.

CHAPTER 2
Literature Review

2.1 Data Drift in Machine Learning Systems

Machine learning models are generally developed under the assumption that training and future production data follow similar statistical distributions. However, in practical deployment environments, this assumption often fails due to changing user behavior, seasonal variations, operational modifications, market trends, or external environmental factors. Such distributional changes are referred to as data drift and are among the primary causes of performance degradation in deployed machine learning systems [1][12].

A model that performs accurately during training may gradually become ineffective when exposed to updated data distributions. This problem is especially critical in applications such as fraud detection, recommendation systems, healthcare analytics, customer churn prediction, and financial forecasting, where inaccurate predictions may lead to severe business or operational consequences [3][14]. In many real-world systems, even small shifts in data patterns can significantly reduce prediction reliability, making continuous monitoring an essential requirement rather than an optional enhancement.

Modern Machine Learning Operations (MLOps) frameworks increasingly emphasize continuous monitoring and maintenance of deployed models to address these issues [16][17][18]. Research in this domain highlights the need for automated monitoring systems capable of identifying distributional changes and triggering corrective actions such as retraining or recalibration [1][13]. Techniques such as statistical drift detection, threshold-based alert generation, and adaptive retraining strategies are now widely considered important components of reliable machine learning pipelines.

It is important to distinguish between data drift and concept drift, as both affect machine learning model performance differently. Data drift refers to changes in the

distribution of input features over time, while concept drift occurs when the relationship between input features and target outcomes changes [1][12]. In practical machine learning systems, both forms of drift may occur simultaneously and contribute to model degradation. The proposed framework in this dissertation primarily focuses on detecting feature-level data drift using statistical methods while evaluating its impact on prediction performance and retraining behavior.

This dissertation focuses on developing a framework that combines drift detection, performance monitoring, and adaptive retraining to improve the robustness of machine learning systems operating in dynamic environments. The proposed system aims to evaluate changes in feature distributions using statistical methods, analyze the impact of drift on model performance, and provide mechanisms for maintaining prediction accuracy over time. By integrating monitoring and maintenance processes into the machine learning lifecycle, the framework seeks to support the development of more stable, scalable, and trustworthy AI systems for real-world deployment scenarios [15][18].

2.2 Statistical Drift Detection Techniques

Statistical drift detection techniques are widely used to identify distributional changes between baseline and incoming datasets. These methods compare historical and current data distributions and generate quantitative indicators of drift severity [1][13].

Effective drift detection techniques should satisfy the following characteristics:

1. Ability to identify distributional changes accurately.
2. Low computational complexity for continuous monitoring.
3. Interpretability for practical deployment scenarios.
4. Compatibility with real-world machine learning pipelines.

Table 2.1: Comparative Analysis of Drift Detection Techniques

Technique	Strength	Limitation
Kolmogorov–Smirnov (KS) Test	Sensitive to distributional changes and statistically rigorous	Interpretation may be difficult for business applications
Population Stability Index (PSI)	Easy to interpret and suitable for threshold-based monitoring	Depends on binning strategy and bucket selection
Jensen–Shannon Divergence	Symmetric and effective for probability distribution comparison	Higher computational complexity
Chi-Square Test	Suitable for categorical feature comparison	Sensitive to sample size and category distribution
Wasserstein Distance	Measures overall distributional movement effectively	Computationally more expensive for large datasets

The comparative analysis shows that different drift detection techniques provide varying trade-offs between statistical sensitivity, interpretability, and computational complexity [1][13][14]. The proposed framework primarily uses KS Test and PSI because they provide an effective balance between statistical reliability, practical interpretability, and computational efficiency for real-world machine learning monitoring systems.

2.2.1 Kolmogorov–Smirnov (KS) Test

The KS Test is a non-parametric statistical method used to compare two probability distributions. It measures the maximum distance between the cumulative distribution functions (CDFs) of two datasets and determines whether the distributions differ significantly [4][5].

A low p-value obtained from the KS Test indicates statistically significant distributional differences between datasets. The method is particularly useful for numerical feature analysis because it does not assume any specific underlying distribution.

However, the method primarily indicates statistical significance and does not directly quantify practical business impact. Therefore, it is often combined with additional metrics such as PSI for improved interpretability.

2.2.2 Population Stability Index (PSI)

The PSI is a statistical metric used to measure the magnitude of distributional changes between two datasets. Unlike the KS Test, PSI provides an interpretable numerical estimate of drift severity, making it highly suitable for production monitoring systems [13].

PSI is calculated by dividing data into bins and comparing the proportion of observations in each bin between baseline and current datasets. Higher PSI values indicate larger distributional differences.

In this dissertation, PSI is used as the primary metric for drift severity assessment.

2.3 MLOps and Model Monitoring

Machine Learning Operations (MLOps) is a discipline that focuses on managing the lifecycle of machine learning systems in production environments [16][17]. Traditional machine learning workflows mainly emphasize model development and evaluation, whereas MLOps extends this process to deployment, monitoring, maintenance, and continuous improvement. By integrating software engineering, automation, and data engineering practices, MLOps enables organizations to deploy machine learning models more efficiently while ensuring long-term reliability and scalability.

One of the major challenges addressed by MLOps is model degradation caused by changing data distributions. Production models often experience performance

deterioration after deployment because incoming data no longer matches training distributions [3][18]. This issue becomes more significant in dynamic environments where user behavior, market conditions, or operational processes continuously evolve. Without proper monitoring mechanisms, degraded models may continue generating inaccurate predictions, leading to poor decision-making and reduced business value.

To overcome these challenges, MLOps frameworks incorporate automated monitoring and maintenance strategies throughout the model lifecycle [15][16]. Monitoring frameworks help organizations detect issues early and reduce risks associated with unreliable predictions. These systems typically track statistical changes in input features, monitor prediction quality metrics, and generate alerts when predefined thresholds are exceeded. In addition, visualization dashboards are commonly integrated into monitoring systems to provide real-time insights into drift behavior, model performance trends, and system health, enabling faster diagnosis and corrective action by data science teams.

Modern MLOps practices also emphasize automation and reproducibility through continuous integration and continuous deployment (CI/CD) pipelines for machine learning systems [16][17]. Such pipelines support automated retraining, validation, and redeployment of models whenever significant drift or performance decline is detected. This reduces manual intervention and helps maintain model effectiveness in rapidly changing environments.

The framework proposed in this dissertation aligns with MLOps principles by combining drift detection, monitoring, visualization, and adaptive retraining into a unified workflow. The proposed approach aims to improve the robustness and maintainability of deployed machine learning systems while supporting continuous performance evaluation in dynamic real-world scenarios.

2.4 Adaptive Retraining Approaches

Adaptive retraining refers to the process of updating machine learning models when performance degradation or significant drift is detected [1][2]. Retraining allows models

to learn updated patterns from new data distributions and improve adaptability under changing conditions.

Several retraining strategies have been proposed in literature, including:

1. Periodic retraining at fixed intervals
2. Threshold-based retraining triggered by drift metrics
3. Online learning approaches
4. Incremental model updates

Threshold-based retraining is widely used because it enables automated model maintenance without requiring continuous manual intervention [2][13]. In such systems, retraining is initiated when drift metrics exceed predefined thresholds.

However, research also highlights that retraining does not always guarantee complete recovery of baseline performance. Excessive drift, noisy data, or unstable label distributions may reduce retraining effectiveness [1][18]. Therefore, drift severity assessment and controlled retraining strategies are critical for maintaining stable machine learning systems.

The proposed framework implements a PSI-based adaptive retraining mechanism, where retraining is automatically triggered when significant drift is detected. This approach enables the system to maintain performance under evolving data conditions while supporting continuous monitoring and analysis.

CHAPTER 3
Project Schema

3.1 Proposed Framework Architecture

The proposed framework is designed to monitor, detect, and adaptively respond to data drift in machine learning systems. The architecture integrates data preprocessing, baseline model training, drift simulation, statistical drift detection, performance evaluation, adaptive retraining, and visualization into a unified workflow.

Initially, historical baseline data is collected and preprocessed using standard data cleaning techniques such as handling missing values, encoding categorical variables, and feature scaling. A machine learning model is then trained using the processed baseline dataset and evaluated using classification performance metrics including accuracy, precision, recall, and F1-score.

To simulate real-world deployment scenarios, controlled drift is introduced into selected numerical and categorical features. The drifted dataset is then analyzed using statistical drift detection methods such as the Kolmogorov–Smirnov (KS) Test and Population Stability Index (PSI).

The framework continuously monitors drift severity and evaluates its impact on model performance. If the detected drift exceeds predefined thresholds, the system automatically triggers adaptive retraining using updated data distributions. Visualization components are also integrated using Plotly dashboards to support monitoring and interpretation of drift behavior.

The overall architecture of the proposed framework is illustrated in Figure (1.1).

3.2 Drift Detection

Drift detection is one of the core components of the proposed framework. The objective of this module is to identify distributional changes between baseline and incoming datasets using statistical techniques. Detecting drift is essential because machine learning models trained on historical data may gradually lose predictive capability when exposed to evolving data distributions in real-world deployment environments.

The proposed framework uses two primary drift detection methods:

1. Kolmogorov–Smirnov (KS) Test
2. Population Stability Index (PSI)

These methods were selected due to their statistical effectiveness, computational efficiency, and practical interpretability for machine learning monitoring systems.

The Kolmogorov–Smirnov (KS) Test is a non-parametric statistical method used to compare the cumulative distributions of two datasets. In the proposed framework, the KS Test is applied to compare baseline feature distributions with incoming drifted feature distributions.

The KS statistic is mathematically represented as:

$$D = \sup_x | F_1(x) - F_2(x) |$$

Where:

- $F_1(x)$ represents the cumulative distribution function of the baseline dataset
- $F_2(x)$ represents the cumulative distribution function of the incoming or drifted dataset
- D represents the maximum distance between the two distributions

A larger KS statistic indicates greater distributional difference between the compared datasets. The associated p-value is used to determine whether the observed difference is statistically significant. A low p-value indicates the presence of statistical drift.

The KS Test is operationally useful because it:

- Does not assume any predefined data distribution
- Works effectively for numerical feature comparison
- Provides statistical significance for drift validation
- Is computationally efficient for monitoring applications

In the proposed framework, the KS Test is primarily used to identify statistical drift in important numerical features such as tenure, MonthlyCharges, and TotalCharges.

In addition to statistical significance testing, the framework uses the Population Stability Index (PSI) to quantify the magnitude of drift between datasets. PSI provides practical interpretability and supports severity-based drift evaluation.

The PSI formula is given by:

$$PSI = \sum(Actual\% - Expected\%) \times \ln\left(\frac{Actual\%}{Expected\%}\right)$$

Where:

- $Expected\%$ represents the proportion of observations in the baseline dataset
- $Actual\%$ represents the proportion of observations in the incoming or drifted dataset

Higher PSI values indicate greater distributional drift severity.

PSI is operationally valuable because it:

- Provides interpretable drift severity estimates
- Supports threshold-based monitoring systems
- Enables automated retraining decisions
- Is widely adopted in industrial monitoring applications

Within the proposed framework, PSI values are used as the primary trigger mechanism for adaptive retraining. The detailed PSI threshold classification and retraining strategy are discussed in Section 3.3.

The combination of KS Test and PSI enables both statistical validation and practical interpretation of drift behavior. This improves the reliability of the proposed monitoring framework under changing data conditions while supporting systematic evaluation of model robustness and adaptive retraining effectiveness.

3.3 Adaptive Retraining Mechanism

The framework incorporates an adaptive retraining mechanism that automatically updates the machine learning model when significant drift is detected.

The retraining decision is based on PSI threshold values. Drift severity is categorized into three levels:

Table 3.1: Drift Severity Classification based on PSI Thresholds

PSI Range	Drift Severity
PSI < 0.1	Low Drift
0.1 ≤ PSI < 0.25	Moderate Drift
PSI ≥ 0.25	High Drift

If any important feature exceeds the predefined high-drift threshold, the framework triggers retraining using the updated drifted dataset.

The retraining process consists of the following steps:

1. Detection of significant drift
2. Drift severity evaluation
3. Trigger generation
4. Model retraining using updated data
5. Post-retraining evaluation

The retrained model is evaluated using the same performance metrics as the baseline model to analyze recovery under drifted conditions.

In addition to restoring prediction performance, the adaptive retraining mechanism helps maintain the stability and relevance of the machine learning system over time. By continuously incorporating updated data into the training process, the framework reduces the risk of outdated decision patterns and improves the model's ability to adapt to evolving data characteristics. The automated retraining workflow also minimizes manual monitoring efforts and supports faster recovery from performance degradation. Such adaptive maintenance approaches are particularly valuable in real-world applications where data patterns frequently change, including financial analytics, customer behavior prediction, fraud detection, and recommendation systems.

3.4 Data Management and Experimental Setup

The experimental setup for the proposed framework was implemented using Python-based machine learning and data analysis libraries including Pandas, NumPy, Scikit-learn, SciPy, Matplotlib, and Plotly. These libraries were used for data preprocessing, model training, statistical drift detection, performance evaluation, and interactive visualization.

Dataset Description

The experimental evaluation of the proposed framework was conducted using a publicly available telecommunications customer churn dataset. The dataset contains customer demographic information, service usage details, billing information, and churn status labels used for classification analysis.

The dataset consists of approximately 7,000 customer records with 21 features, including both numerical and categorical variables. Important features include tenure, MonthlyCharges, TotalCharges, Contract, PaymentMethod, InternetService, and Churn.

The target variable, Churn, represents whether a customer discontinued the service. The dataset contains both churn and non-churn customer instances, enabling supervised machine learning analysis for churn prediction.

The telecommunications churn dataset is particularly suitable for drift analysis because customer behavior in telecom environments changes frequently due to:

- pricing modifications,
- evolving service plans,
- market competition,
- customer retention strategies,
- changing usage patterns,
- and economic conditions.

Such dynamic behavior naturally leads to distributional changes in customer-related features over time, making churn prediction systems highly susceptible to data drift.

Additionally, the dataset contains a combination of behavioral, financial, and service-related attributes, providing a realistic environment for evaluating statistical drift detection techniques and adaptive retraining strategies.

The diversity of feature types and changing customer behavior patterns make the dataset well suited for experimentation related to drift monitoring and machine learning system maintenance.

The study uses a customer churn prediction dataset containing both numerical and categorical features. Data preprocessing includes:

- Handling missing values
- Encoding categorical variables
- Feature selection
- Data normalization where required

The dataset was divided into baseline and drifted versions to simulate real-world deployment conditions. Controlled feature drift and label drift were introduced to evaluate model robustness under changing data distributions.

Feature Drift Simulation

Feature drift was introduced by modifying the statistical distribution of important numerical features in the incoming dataset. The selected features for drift injection were:

- tenure
- MonthlyCharges
- TotalCharges

These features were chosen because they strongly influence customer churn prediction and showed measurable sensitivity during preliminary analysis.

For the **tenure** feature, customer subscription duration values were increased to simulate evolving customer retention behavior over time.

For **MonthlyCharges**, controlled scaling and shifting operations were applied to represent changes in billing patterns, service pricing, and subscription plans.

Similarly, **TotalCharges** values were adjusted proportionally to simulate cumulative spending variations caused by changing customer behavior.

The drift injection strategy intentionally modified feature distributions while preserving the overall dataset structure and realistic relationships between variables. This enabled systematic evaluation of drift detection methods without introducing excessive random noise.

Label Drift Simulation

In addition to feature drift, label drift was introduced by modifying the class distribution of the target variable (Churn). Selected instances were relabeled to increase the proportion of churn cases within the drifted dataset.

This simulated changing customer churn behavior commonly observed in real-world business environments due to market competition, service dissatisfaction, pricing changes, or external economic conditions.

The modified target distribution enabled analysis of model robustness under evolving class imbalance conditions.

Importance of Controlled Drift Simulation

Controlled drift simulation provides several advantages for experimental evaluation:

1. Enables reproducible experimentation under known drift conditions
2. Allows systematic comparison between baseline and drifted performance
3. Supports validation of statistical drift detection techniques
4. Enables evaluation of adaptive retraining effectiveness

In practical production systems, drift occurs naturally and unpredictably over time. However, real-world deployment data is often unavailable or difficult to continuously monitor within academic environments. Therefore, controlled simulation provides a practical and reliable approach for evaluating drift-aware machine learning systems under realistic conditions.

The baseline machine learning model was trained using Random Forest classification and evaluated using accuracy, precision, recall, and F1-score metrics.

Selection of Random Forest Classifier

Random Forest classification was selected as the baseline machine learning model due to its robustness, stability, and suitability for heterogeneous tabular datasets containing both numerical and categorical features.

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting. The algorithm performs effectively on structured business datasets and requires relatively low preprocessing complexity compared to several other machine learning models.

The model was considered suitable for the proposed framework for the following reasons:

1. Ability to handle both numerical and categorical features effectively
2. Robustness against noise and moderate data imbalance
3. Reduced risk of overfitting through ensemble averaging
4. Stable baseline performance for comparative drift analysis
5. Interpretability through feature-level analysis and performance metrics

Additionally, Random Forest provides consistent performance across varying data distributions, making it appropriate for evaluating the impact of drift and retraining within the proposed monitoring framework.

Table 3.2: Major dataset features used in the experimental setup

Feature	Type	Purpose in Analysis
customerID	Identifier	Unique customer identifier
gender	Categorical	Customer demographic information
SeniorCitizen	Categorical	Age-group related customer behavior
Partner	Categorical	Relationship status analysis
Dependents	Categorical	Household dependency analysis
tenure	Numerical	Customer subscription duration
InternetService	Categorical	Type of internet service used
OnlineSecurity	Categorical	Service usage behavior
Contract	Categorical	Customer contract type
PaymentMethod	Categorical	Payment behavior analysis
MonthlyCharges	Numerical	Monthly billing information
TotalCharges	Numerical	Overall customer spending
Churn	Target Variable	Customer churn prediction outcome

The experiments were conducted in a Jupyter Notebook environment, while interactive visualizations and monitoring dashboards were implemented using Plotly. The overall framework enables systematic analysis of drift behavior and adaptive model maintenance under evolving data conditions.

CHAPTER 4
Results

This chapter presents the experimental results obtained from the proposed framework for monitoring, detecting, and adaptively responding to data drift in machine learning systems. The experiments focus on evaluating the impact of drift on model performance, analyzing drift severity using statistical methods, and assessing the effectiveness of adaptive retraining mechanisms.

The baseline machine learning model was initially trained using the original dataset and evaluated using standard classification metrics including accuracy, precision, recall, and F1-score. Controlled feature drift and label drift were then introduced into the dataset to simulate real-world distributional changes.

The performance of the machine learning model was analyzed under three different stages:

1. Baseline model performance
2. Performance under drifted conditions
3. Performance after adaptive retraining

The results obtained from these experiments are discussed in the following sections.

4.1 Baseline Model Performance

The baseline model was trained using the original dataset before introducing any drift. The model was evaluated using accuracy, precision, recall, and F1-score metrics.

Table 4.1: Baseline model performance metrics

Metric	Value
Accuracy	0.788152
Precision	0.658711

Metric	Value
Recall	0.475862
F1 Score	0.552553

The baseline model achieved satisfactory classification performance on the original dataset. Accuracy and precision values indicate stable prediction capability under normal conditions. However, recall values were comparatively lower, indicating moderate limitations in identifying all positive churn cases.

Evaluation Metric Interpretation

In customer churn prediction systems, evaluation metrics such as recall and F1-score are particularly important because the target classes are often imbalanced. In such scenarios, relying solely on accuracy may produce misleading conclusions regarding model effectiveness.

Recall measures the ability of the model to correctly identify actual churn cases. A low recall value indicates that a significant number of churn customers are incorrectly classified as non-churn customers. These false negatives are critical in churn prediction because undetected churn customers may lead to customer loss, reduced revenue, and ineffective retention strategies.

Precision evaluates the proportion of correctly predicted churn cases among all predicted churn instances. While higher precision reduces false alarms, excessive emphasis on precision may increase false negatives and reduce recall.

The F1-score provides a balanced measure between precision and recall and is particularly useful when class distributions are imbalanced. In the proposed framework, F1-score is used to evaluate overall classification stability under drifted conditions.

The experimental analysis demonstrates that data drift significantly impacts recall and F1-score, indicating deterioration in the model's ability to reliably detect churn behavior under changing data distributions.

4.2 Impact of Drift on Model Performance

To simulate real-world deployment scenarios, controlled drift was introduced into selected numerical and categorical features. Statistical distributional changes were applied to important features such as tenure, MonthlyCharges, and TotalCharges.

The drifted dataset was evaluated using the baseline model to analyze the impact of drift on prediction performance.

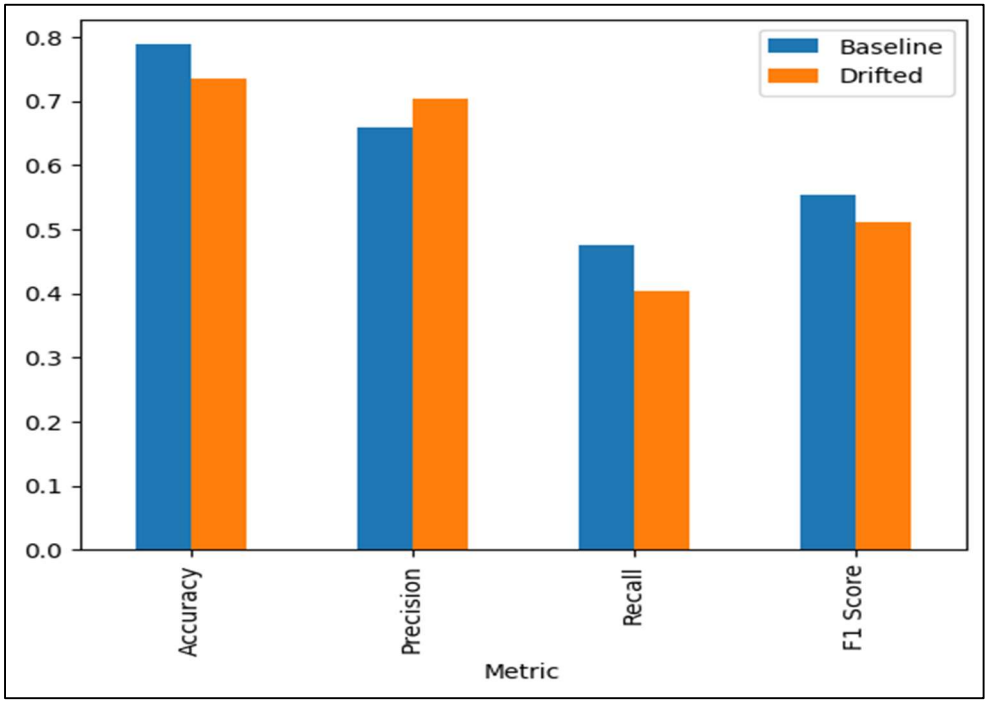


Figure 4.1: Comparison of baseline and drifted model performance across evaluation metrics.

The results demonstrate that drift significantly affects machine learning model reliability. Accuracy, recall, and F1-score decreased under drifted conditions, indicating reduced predictive effectiveness. Among all metrics, recall experienced the largest decline, suggesting that the model became less effective at identifying churn cases after distributional changes.

Interestingly, precision increased slightly under drifted conditions. This indicates that the model became more conservative in predicting positive churn cases, resulting in fewer false positives but increased false negatives.

These findings highlight the importance of continuous monitoring in deployed machine learning systems.

4.3 Drift Detection Analysis

The proposed framework uses the Kolmogorov–Smirnov (KS) Test and Population Stability Index (PSI) to identify and quantify drift severity.

Table 4.2: Drift Detection Summary using KS Test and PSI

Feature	KS Statistic	p-value	PSI
tenure	0.181	6.13e-31	0.093
MonthlyCharges	0.293	1.04e-80	0.300
TotalCharges	0.081	1.89e-06	0.027

The KS Test identified statistically significant distributional changes in all analyzed numerical features, indicating that the distributions of incoming data differed noticeably from the original training data. The extremely small p-values obtained for all features confirm that these differences were unlikely to have occurred by random variation alone. This demonstrates the sensitivity of the KS Test in detecting even subtle shifts in feature distributions within large datasets.

However, PSI provided more practical insight into drift severity by quantifying the magnitude of distributional change rather than only confirming its statistical existence. While the KS Test indicates whether drift is present, PSI helps determine whether the detected drift is operationally important enough to impact model reliability. This distinction is important in real-world monitoring systems because statistically significant drift may not always lead to meaningful performance degradation. PSI therefore supports

better decision-making by categorizing drift into interpretable severity levels such as low, moderate, and high drift.

The combined use of KS Test and PSI also improves monitoring reliability by balancing sensitivity and interpretability. The KS Test acts as an early statistical indicator of distributional variation, whereas PSI provides a business-oriented measure that helps prioritize corrective actions such as investigation, alert generation, or retraining. This complementary behavior makes the integrated approach more suitable for production-level machine learning monitoring frameworks.

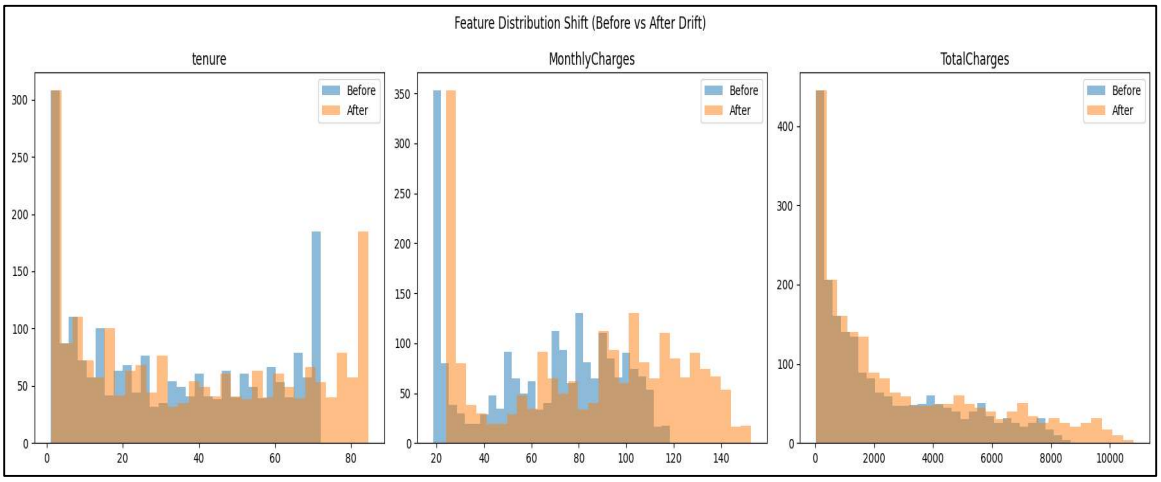


Figure 4.2: Feature distribution comparison before and after drift introduction for major numerical features.

MonthlyCharges showed the highest PSI value, indicating significant drift and major distributional change. According to the predefined PSI thresholds, this feature falls into the high-drift category, suggesting that changes in customer billing patterns strongly affected the incoming data distribution. Such drift can substantially impact model predictions because MonthlyCharges is an important predictor in customer churn analysis.

In contrast, TotalCharges exhibited comparatively lower PSI despite statistical significance in the KS Test. This demonstrates that statistical significance alone may not always indicate severe practical drift. The KS Test is highly sensitive to distributional

differences, especially for large datasets, whereas PSI provides a clearer measure of the actual magnitude of drift. Therefore, PSI is more suitable for operational monitoring and business interpretation.

The tenure feature showed moderate statistical change with a PSI value below the warning threshold, indicating relatively stable behavior compared to MonthlyCharges. This suggests that customer subscription duration remained mostly consistent even after drift simulation, and its influence on prediction stability was comparatively lower.

The experimental results also indicate that combining KS Test and PSI provides a more reliable monitoring strategy than using a single technique alone. The KS Test effectively detects whether a distributional change exists, while PSI helps evaluate whether the detected change is practically important enough to require intervention. This combined interpretation improves decision-making in monitoring systems by reducing false alarms and prioritizing significant drift conditions.

Overall, the results confirm that PSI provides improved interpretability for real-world monitoring systems. The findings also validate the effectiveness of the proposed framework in identifying feature-level drift and supporting adaptive maintenance decisions in dynamic machine learning environments.

4.4 Adaptive Retraining Evaluation

When drift severity exceeded predefined thresholds, the framework automatically triggered model retraining using the updated drifted dataset.

The retrained model was evaluated using the same classification metrics to analyze recovery under drifted conditions. This comparison provides a clearer understanding of how data drift affects predictive performance and how effectively retraining restores model reliability.

Table 4.3: Comparative analysis of baseline, drifted, and retrained model performance.

=== FINAL PERFORMANCE COMPARISON ===

	Metric	Baseline	Drifted	Retrained	Drop (Baseline→Drifted)	Recovery (Drifted→Retrained)
0	Accuracy	0.788152	0.735071	0.685624	0.053081	-0.049447
1	Precision	0.658711	0.702638	0.707317	-0.043927	0.004679
2	Recall	0.475862	0.402473	0.434457	0.073390	0.031984
3	F1 Score	0.552553	0.511790	0.538283	0.040762	0.026493

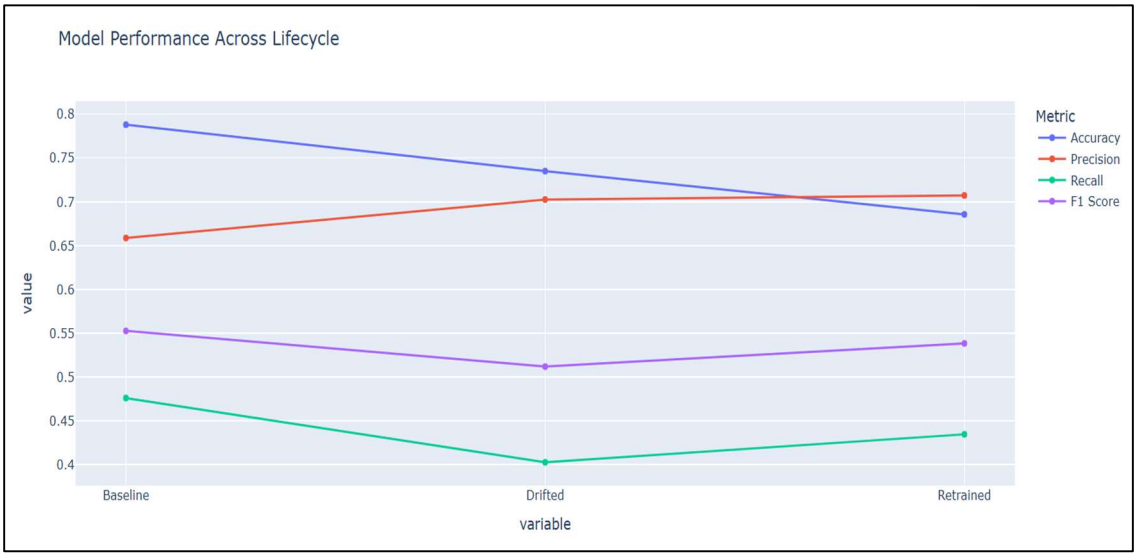


Figure 4.3: Performance comparison across baseline, drifted, and retrained stages

The results show that adaptive retraining improved model stability under drifted conditions. Recall and F1-score showed noticeable recovery after retraining, indicating improved adaptability to updated data distributions.

Although retraining did not fully restore baseline accuracy, the framework successfully mitigated performance degradation caused by drift. This highlights the practical importance of adaptive monitoring and retraining strategies in dynamic machine learning environments.

The experiments also demonstrate that relying solely on accuracy may provide misleading conclusions under drifted conditions. Metrics such as recall and F1-score provide better insight into model robustness and prediction reliability.

4.5 Interactive Dashboard Visualization

To improve interpretability and monitoring capability, an interactive dashboard was developed using the Plotly visualization library. The dashboard serves as the visualization and monitoring interface of the proposed framework and enables real-time interpretation of drift severity and model behavior.

The dashboard integrates multiple visualization components designed to support operational monitoring of machine learning systems under changing data conditions. The primary dashboard components include:

- PSI-based drift severity visualization
- Threshold highlighting for drift classification
- Feature-level drift monitoring
- Model performance comparison graphs
- Performance recovery visualization after retraining

The PSI threshold visualization component highlights drift severity levels using predefined threshold ranges. Features exceeding moderate or high drift thresholds can be identified quickly, enabling faster monitoring and decision-making.

The dashboard also supports feature-level monitoring by visualizing distributional changes between baseline and drifted datasets. Important numerical features such as tenure, MonthlyCharges, and TotalCharges are monitored to identify major contributors to drift.

In addition to drift monitoring, the dashboard provides comparative visualization of model performance across baseline, drifted, and retrained stages. This enables analysis of

performance degradation and retraining effectiveness using metrics such as accuracy, precision, recall, and F1-score.

The interactive visualization framework improves operational usability by simplifying interpretation of statistical drift metrics and supporting continuous monitoring of deployed machine learning systems. Such monitoring dashboards are particularly valuable in production environments where early detection of model degradation is critical for maintaining prediction reliability.

Figure 4.4 illustrates the interactive dashboard developed as part of the proposed framework.

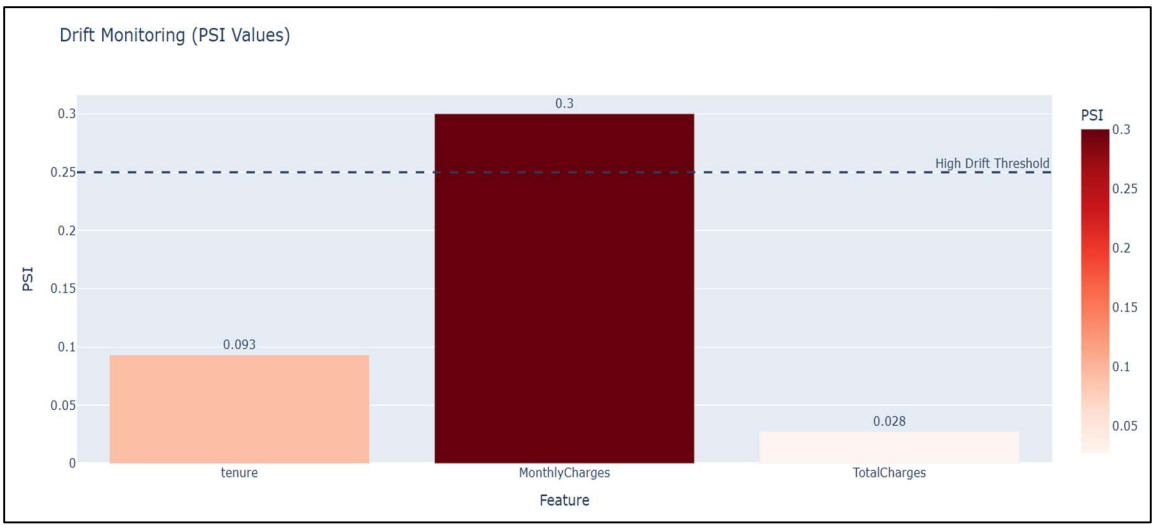


Figure 4.4: Interactive dashboard for monitoring drift severity and model performance.

4.6 Summary of Experimental Observations

Table 4.4: Summary of Key Experimental Observations

Observation	Interpretation
Recall decreased significantly under drift	Drift strongly affected churn detection capability
MonthlyCharges showed highest PSI	Major feature-level drift detected

Observation	Interpretation
Retraining improved recall and F1-score	Adaptive retraining improved robustness
Accuracy alone did not reflect complete impact	Multiple evaluation metrics are necessary

Table 4.4 summarizes the major experimental findings obtained during the implementation and evaluation of the proposed drift monitoring framework. The observations collectively demonstrate how distributional changes influence model behavior and how adaptive retraining contributes to maintaining predictive stability in dynamic environments.

The significant decrease in recall under drifted conditions indicates that the model became less effective at correctly identifying churn instances after exposure to changed data distributions. Since recall measures the proportion of actual positive cases correctly detected by the model, the decline suggests that many churn cases were missed when the original model encountered unfamiliar patterns in the drifted dataset. In practical business scenarios, this can lead to major operational consequences because undetected churn customers may leave without receiving retention interventions. The observation therefore highlights that drift directly affects the model's capability to capture important behavioral patterns associated with customer attrition.

The observation that MonthlyCharges showed the highest PSI value confirms the presence of substantial feature-level drift within the dataset. A high PSI indicates that the statistical distribution of this feature changed considerably compared to the baseline training distribution. This suggests that billing-related customer behavior evolved more aggressively than other monitored variables. Such changes may result from revised pricing policies, promotional campaigns, subscription modifications, or shifts in customer usage trends. Since MonthlyCharges is an important predictive feature in churn analysis, large variations in its distribution can significantly influence overall model predictions and contribute to performance degradation.

Another important finding is that adaptive retraining improved recall and F1-score after drift detection. This demonstrates that retraining enabled the model to learn updated patterns present in the new data distribution and partially recover lost predictive capability. Improvement in recall indicates better detection of churn cases after retraining, while recovery in F1-score reflects a more balanced relationship between precision and recall. These improvements validate the effectiveness of the proposed adaptive framework in maintaining model robustness under changing environmental conditions. The results also suggest that integrating monitoring with automated retraining can reduce long-term degradation in deployed machine learning systems.

The experiments additionally revealed that relying solely on accuracy may provide misleading conclusions regarding model reliability. In several cases, accuracy remained relatively stable even though recall and F1-score deteriorated considerably. This occurs because accuracy evaluates only the overall proportion of correct predictions and may fail to capture poor performance on minority or high-priority classes. In customer churn prediction, correctly identifying churn cases is often more important than maximizing overall accuracy. Therefore, using multiple evaluation metrics provides a more comprehensive understanding of model behavior under drifted conditions. Metrics such as recall, precision, and F1-score offer deeper insight into prediction quality and robustness than accuracy alone.

The observations summarized in the table also demonstrate the importance of feature-level monitoring in practical machine learning systems. Drift does not necessarily affect all features equally, and certain variables may contribute more strongly to model instability. Identifying high-drift features enables organizations to prioritize corrective actions more effectively and improve monitoring efficiency. This supports the development of targeted retraining strategies rather than relying on unnecessary full-system updates.

Feature-level monitoring also helps reduce unnecessary computational overhead in machine learning maintenance workflows. Instead of retraining the entire system whenever minor drift occurs, organizations can focus monitoring efforts on features

exhibiting significant distributional changes. This improves resource efficiency and enables more targeted corrective actions. Additionally, identifying high-drift features provides better diagnostic insight into the underlying causes of model degradation, helping analysts understand which business or behavioral factors contribute most strongly to performance instability. Such targeted monitoring approaches are particularly valuable in large-scale production environments where continuous full-system retraining may be computationally expensive and operationally inefficient.

Overall, the experimental results demonstrate that data drift significantly impacts machine learning model reliability and that continuous monitoring combined with adaptive retraining helps improve model robustness under changing data conditions. The findings validate the practical usefulness of the proposed framework and emphasize the importance of integrating drift detection, performance evaluation, and retraining mechanisms into modern MLOps-based machine learning systems.

CHAPTER 5
Conclusion and Future Scope

5.1 Conclusion

Machine learning systems deployed in real-world environments are highly vulnerable to changes in data distribution over time. Such changes, commonly referred to as data drift, can significantly reduce model reliability and prediction quality if not detected and addressed appropriately.

This dissertation presented a framework for monitoring, detecting, and adaptively responding to data drift in machine learning systems. The proposed framework integrates statistical drift detection techniques, performance monitoring, visualization, and adaptive retraining into a unified workflow.

Controlled feature drift and label drift were introduced into the dataset to simulate realistic deployment conditions. The experimental results demonstrated that drift significantly affects machine learning model performance, particularly reducing recall and overall prediction reliability. Among the evaluated features, MonthlyCharges exhibited the highest PSI value, indicating major distributional drift.

The study also demonstrated that statistical significance alone does not always indicate severe practical drift. While the KS Test successfully identified distributional changes, PSI provided improved interpretability for evaluating drift severity and retraining decisions.

An adaptive retraining mechanism was implemented to improve model robustness under drifted conditions. Experimental results showed that retraining partially recovered degraded performance metrics, particularly recall and F1-score, although baseline accuracy was not fully restored. These findings highlight the practical importance of

continuous monitoring and adaptive maintenance strategies in dynamic machine learning environments.

To improve interpretability and usability, an interactive dashboard was developed using Plotly for visualization of drift severity, feature distribution changes, and model performance trends. The dashboard enhances monitoring capability and supports practical analysis of machine learning system behavior under evolving data distributions.

Overall, the proposed framework successfully demonstrates how monitoring, statistical drift detection, adaptive retraining, and visualization can be integrated to maintain reliability of machine learning systems in changing environments.

The findings of this dissertation also highlight the growing operational importance of continuous monitoring in production-level machine learning systems. Modern AI applications increasingly operate in dynamic environments where customer behavior, market conditions, and data patterns evolve continuously over time. In such scenarios, static machine learning models may gradually become unreliable if distributional changes are not detected and addressed appropriately.

The proposed framework demonstrates how drift detection, performance monitoring, visualization, and adaptive retraining can be integrated into broader MLOps-based machine learning workflows. Incorporating automated monitoring mechanisms helps organizations identify model degradation early, reduce long-term prediction instability, and improve system reliability under changing operational conditions.

The study further emphasizes that maintaining deployed machine learning systems requires ongoing monitoring and maintenance rather than one-time model development. As machine learning adoption continues to expand across industries, drift-aware monitoring frameworks are expected to become increasingly important components of scalable and trustworthy AI systems.

Limitations of the Study

Although the proposed framework successfully demonstrates drift detection and adaptive retraining under controlled conditions, the study contains certain limitations. The experiments were conducted using simulated drift scenarios rather than naturally evolving real-time production data. While controlled drift simulation improves reproducibility and systematic evaluation, real-world drift behavior may exhibit greater complexity and unpredictability.

Additionally, the current framework operates using a batch-based experimental setup instead of continuous real-time streaming data environments. As a result, the system does not fully evaluate challenges associated with live deployment, such as streaming latency, online learning constraints, and continuous model updating.

The study also focuses primarily on feature-level data drift and does not extensively address advanced forms of concept drift or large-scale distributed monitoring systems. Despite these limitations, the proposed framework provides a practical and effective foundation for understanding drift-aware machine learning monitoring and adaptive retraining strategies.

5.2 Future Scope

Although the proposed framework effectively demonstrates drift detection and adaptive retraining under controlled experimental conditions, several improvements and extensions can be explored in future work.

1. **Real-Time Drift Monitoring:**
   The current implementation uses batch-based analysis. Future work can extend the framework to real-time streaming environments using technologies such as Apache Kafka or Apache Spark Streaming.

2. **Advanced Drift Detection Techniques:**
   Additional drift detection methods such as Jensen–Shannon Divergence,

Wasserstein Distance, or machine learning-based drift detectors can be incorporated for improved analysis.

3. **Online and Incremental Learning:**
   Instead of full retraining, future systems may implement online learning approaches where models continuously update using incoming data streams.

4. **Automated MLOps Integration:**
   The framework can be integrated with production-level MLOps pipelines for automated deployment, monitoring, retraining, and version management.

5. **Multi-Model Monitoring Systems:**
   Future research can extend the framework to monitor multiple deployed models simultaneously within enterprise-scale environments.

6. **Enhanced Dashboard Functionality:**
   Additional dashboard features such as real-time alerts, drift trend forecasting, and automated reporting can improve monitoring capability and operational usability.

7. **Domain-Specific Deployment:**
   The framework can be adapted for domain-specific applications such as healthcare analytics, fraud detection, cybersecurity, and financial risk assessment where data drift is highly critical.

The proposed framework provides a strong foundation for future research and development in reliable and adaptive machine learning system monitoring.

Bibliography

[1] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.

[2] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448, Minneapolis, Minnesota, USA, 2007.

[3] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pages 2503–2511, Montreal, Canada, 2015.

[4] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edition, Springer, New York, 2009.

[5] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer, New York, 2013.

[6] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 2nd edition, O'Reilly Media, California, 2019.

[7] Andreas C. Müller and Sarah Guido. *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media, California, 2016.

[8] Andriy Burkov. *Machine Learning Engineering*. True Positive Inc., 2020.

[9] Wes McKinney. *Python for Data Analysis*. 3rd edition, O'Reilly Media, California, 2022.

[10] Plotly Technologies Inc. Plotly Python Graphing Library Documentation. Available at: https://plotly.com/python/, accessed April 2026.

[11] Scikit-learn Developers. Scikit-learn: Machine Learning in Python Documentation. Available at: https://scikit-learn.org/, accessed April 2026.

[12] Indrė Žliobaitė. Learning under concept drift: An overview. *arXiv preprint arXiv:1010.4784*, 2010.

[13] Raquel Sebastião and João Gama. A study on change detection methods. In *Progress in Artificial Intelligence*, pages 12–23. Springer, Berlin, Heidelberg, 2009.

[14] Charu C. Aggarwal. *Data Streams: Models and Algorithms*. Springer, New York, 2007.

[15] Emmanuel Ameisen. *Building Machine Learning Powered Applications*. O'Reilly Media, California, 2020.

[16] Mark Treveil and Alok Shukla. *Introducing MLOps: How to Scale Machine Learning in the Enterprise*. O'Reilly Media, California, 2020.

[17] Noah Gift and Alfredo Deza. *Practical MLOps: Operationalizing Machine Learning Models*. O'Reilly Media, California, 2021.

[18] Chip Huyen. *Designing Machine Learning Systems*. O'Reilly Media, California, 2022.

[19] Apache Software Foundation. Apache Kafka Documentation. Available at: https://kafka.apache.org/documentation/, accessed April 2026.